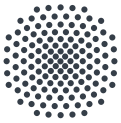

University of Stuttgart
Germany

Advanced Simulation Methods

Machine Learned Interatomic Potentials

Jakob Wallner, Stephan Haag

Supervisor: Fabian Zills, Julian Peters

Version: 0

Contents

1 Theory

1.1 Molecular Dynamics Simulation

Molecular Dynamics (MD) is a computational method used for molecular modeling and simulation in which the interactions between atoms and molecules are iteratively calculated to predict the behavior of a system over time. In order to calculate the forces acting on each atom, a potential energy function V is used, which describes the interactions between atoms. Besides classical force-fields, which are based on empirical formulas, machine learning interatomic potentials (MLIPs) have emerged as a powerful alternative, providing a more accurate representation of the potential energy surface (PES) by learning from quantum mechanical calculations. Commonly a equation of motion has to be solved, which uses Newton's second law of motion:

$$m\frac{d^2\mathbf{r}}{dt^2} = -\nabla V(\mathbf{r}) \tag{1.1}$$

where m is the mass of the atom, $\mathbf{r}$ is the position vector of the atom, and ∇V is the gradient of the potential energy function with respect to the position. With a suitable numerical integration method, such as the Verlet algorithm, the positions and velocities of the atoms can be updated iteratively to simulate the dynamics of the system over time. After a simulation step, the forces acting on each atom are recalculated based on the updated positions, and the process is repeated until the desired simulation time is reached. The simulation typically runs in with the constrains of a specific ensemble, such as the microcanonical (NVE), canonical (NVT) or isothermal-isobaric (NPT) ensemble, which defines the thermodynamic conditions of the system. The ensemble used in the project is a NVT ensemble, where the number of particles N, the volume V and the temperature T are kept constant. This is achieved by using a thermostat, such as the Langevin thermostat, which adds a stochastic term to the equations of motion to control the temperature of the system. Other common thermostats include the Nosé-Hoover thermostat and the Berendsen thermostat, each with its own advantages and disadvantages in terms of accuracy and computational efficiency. The Langevin thermostat, combines deterministic and stochastic forces to maintain the desired temperature. This has a downside of changing the dynamics of the system resulting in a non-newtonian trajectory. Therefore dynamic properties, such as diffusion coefficients, should not be calculated from a NVT simulation with a Langevin thermostat. In order to evaluate thermodynamic properties, the ensemble average (average over all microstates of the system) is calculated. The output of a MD simulation is typically a long trajectory of microstates evolving over time. The ergodic hypthesis connects the time average of a property to its ensemble average, allowing us to compute thermodynamic properties from a single trajectory, provided that the system is ergodic and the simulation time is sufficiently long.

1.2 Quantum Mechanical Foundations

A quantum mechanical system is generally described by a wavefunction Ψ and the time-dependent Schrödinger equation:

$$i\hbar\frac{\partial\Psi}{\partial t} = \hat{H}\Psi \tag{1.2}$$

where $\hbar$ is the reduced Planck constant, and $\hat{H}$ is the Hamiltonian operator, which represents the total energy of the system. For a single particle in a potential $V(\mathbf{r})$, the Hamiltonian can be expressed

as:

$$\hat{H} = -\frac{\hbar^2}{2m}\nabla^2 + V(\mathbf{r}) \tag{1.3}$$

where m is the mass of the particle, and ∇^2 is the Laplacian operator, which accounts for the kinetic energy of the particle. Because this is to complex to solve for many-body systems, the Born-Oppenheimer approximation is often used, which separates the electronic and nuclear degrees of freedom, based on the difference in their masses. This reduces the problem to solving the electronic Schrödinger equation for fixed nuclear positions. Another approximation is the linear combination of atomic orbitals (LCAO), which expresses the molecular orbitals as a linear combination of atomic orbitals. Combined with the Hartree-Fock method, which approximates the many-electron wavefunction as a single Slater determinant, this allows for the calculation of electronic structure and properties of molecules.

In order to simulate a quantum mechanical system, the Density Functional Theory (DFT) is often used, which is a computational quantum mechanical method that describes the electronic structure of many-body systems in terms of the electron density $\rho(\mathbf{r})$ rather than the wavefunction. The Hohenberg-Kohn theorems state that the ground-state properties of a many-electron system are uniquely determined by the electron density, and that there exists a universal functional $E[\rho]$ that can be used to calculate the total energy of the system. The Kohn-Sham equations transform the many-electron problem into a set of single-particle equations in a self-consistent field (SCF) approach, where the electron density is iteratively updated until convergence is achieved. The result is a single particle potential:

$$V_\mathrm{s}(\mathbf{r}) = V_\mathrm{ext}(\mathbf{r}) + V_H(\mathbf{r}) + V_\mathrm{xc}(\mathbf{r}) \tag{1.4}$$

where V_ext is the external potential, V_H is the Hartree potential, and V_xc is the exchange-correlation potential, which accounts for the many-body effects of electron-electron interactions. There are many different exchange-correlation functionals, such as the Local Density Approximation (LDA), the Generalized Gradient Approximation (GGA) and hybrid functionals, which combine DFT with Hartree-Fock exchange.

1.3 Dispersion Correction

Because DFT is a mean-field theory, it fails to capture long-range dispersion interactions, which are crucial for accurately describing the behavior of many systems. In order to account for these interactions, dispersion correction methods such as the Grimme-D3 methods can be applied after the DFT calculation. The D3 method adds an empirical dispersion energy term to the total energy calculated by DFT, which is based on a pairwise additive model of dispersion interactions between atoms. The energy correction is calculated as:

$$E_\mathrm{disp} = -\sum_{i<j}\sum_{N}\left(\frac{C_6^{ij}}{R_{ij,L}^6} f_{d,6}(R_{ij,L}) + \frac{C_8^{ij}}{R_{ij,L}^8} f_{d,8}(R_{ij,L})\right) \tag{1.5}$$

where C_6^{ij} and C_8^{ij} are the dispersion coefficients for the atom pair i, j, $R_{ij,L}$ is the distance between the atoms, and $f_{d,n}(R)$ is a damping function that prevents the correction from diverging at short distances. Dispersion corrections are independent of the underlying electron density.

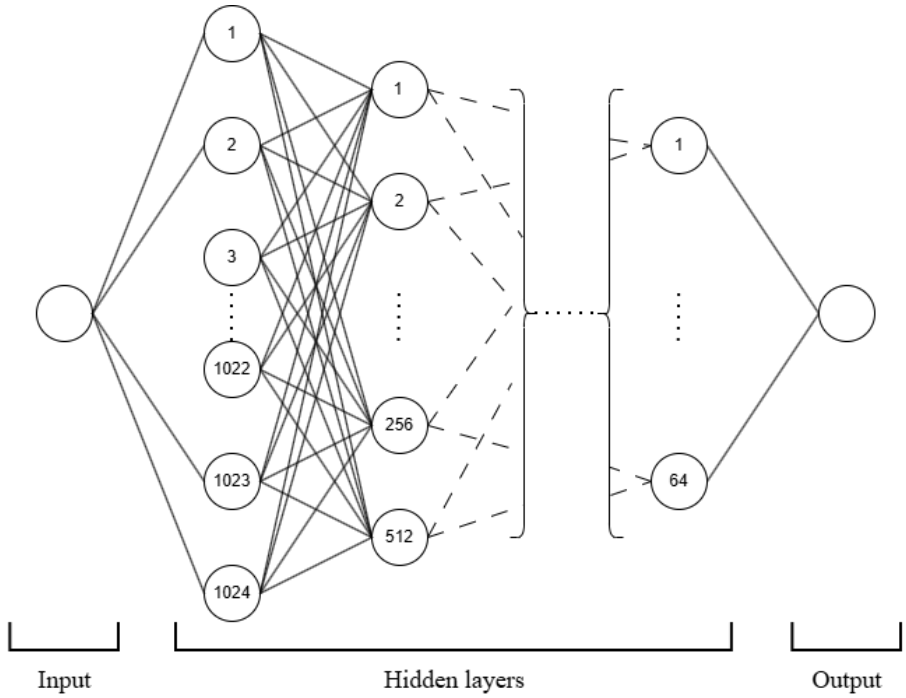

Figure 1: *Schematic representation of a simple feedforward neural network with several hidden layers.*

1.4 Neural Networks

The modern Neural Networks (NNs) are based on a mathematical model of the neuron, a so-called artificial neuron or perceptron. This neuron receives multiple inputs $\mathbf{x}$ and applies a learnable weight w_i to each input, sums them up and adds a bias b to produce an output y. This information is then passed through a non-linear activation function f to introduce non-linearity into the model, allowing it to learn complex patterns in the data.

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(\mathbf{x} \cdot \mathbf{w} + b) \tag{1.6}$$

A common activation function is the Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$. In order to advance in high-dimensional problems, multiple layers of neurons are stacked together to form a deep neural network. The arangement of these layers, called architecture, can vary widely and changes the way the network learns and processes information. Usual architectures include feedforward networks, convolutional neural networks (CNNs) and recurrent neural networks (RNNs).

A better way to represent the architecture of a neural network is through matrix operations. A hidden layer l with m neurons receives an input vector $\mathbf{a}^{(l-1)}$ of size n from the previous layer, applies a weight matrix $\mathbf{W}^{(l)}$ of size $m \times n$ and a bias vector $\mathbf{b}^{(l)}$ of size m, and produces an output vector $\mathbf{a}^{(l)}$ of size m.

$$\mathbf{a}^{(l)} = f\left(\mathbf{W}^{(l)} \cdot \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{1.7}$$

Stacking multiple layers allows the network to become a highly complex and non-linear function. The combination of the non-linear activation functions and the learnable parameters (weights and biases) enables the network to approximate a wide variety of functions, making it a powerful tool for tasks such as classification, regression, and generative modeling.

1.5 Training a Neural Network

In order to improve the performance of a NN, it needs to be trained on a dataset. This process is realized by determining the optimal network parameters (weights $\mathbf{W}$ and biases $\mathbf{b}$) that minimize a loss function $\mathcal{L}$, which quantifies the difference between the predicted output and the true output. A simple example of a loss function is the mean squared error (MSE) for regression tasks:

$$\mathcal{L}_{MSE}(\hat{y}, y) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 \tag{1.8}$$

where $\hat{y}$ is the predicted output, y is the true output, and N is the number of samples in the dataset.

1.6 Descriptors

In order to reduce the complexity of the input data and ensure a consistent input vector size, descriptors are used to represent the atomic environment in a fixed-size vector. A descriptor transforms the atomic environment into a numerical representation that captures the relevant information for the task at hand, such as the local geometry and chemical composition. The Gaussian Moment Neural Network Input descriptor (short:GMNN-descriptor, used in this project) ensures that the input vector remains invariant under:

- Translation: The descriptor does not change if the entire system is shifted in space.
- Rotation: The descriptor remains unchanged if the system is rotated in space.
- Permutation: The descriptor is invariant to the permutation of identical atoms.

This also allows the network to learn the underlying physics of the system, rather than just memorizing specific configurations, which improves the generalization of the model to unseen data.

1.7 Local Atomic Energy Partitioning

A essential of an MLIP is that the energies are calculated on a local atomic level, the total energy of the system is a derived quantity. Behler and Parinello intriduced the concept of local atomic energy partitioning, stating that the total energy of a system can be decomposed into contributions from individual atoms, which depend on their local environment.

$$E_{\text{total}} = \sum_{i=1}^{N} E_i \tag{1.9}$$

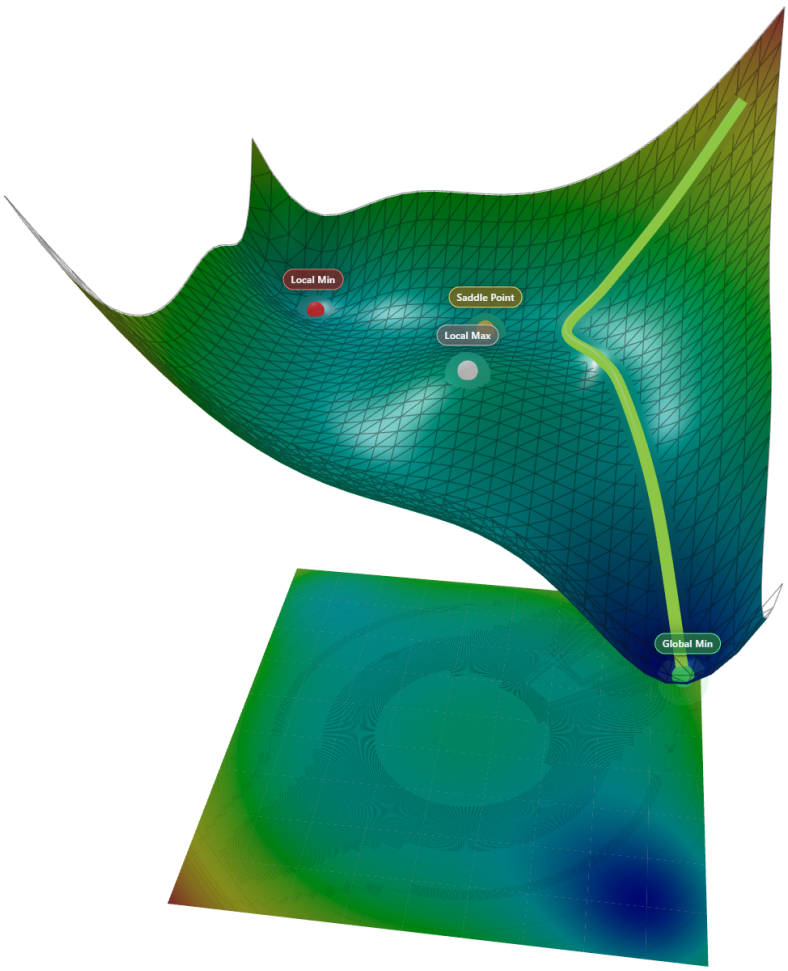

Figure 2: *Schematic representation of the training process of a neural network, where the loss function is minimized by adjusting the weights and biases.*

where E_{total} is the total energy of the system, N is the number of atoms, and E_i is the local atomic energy contribution from atom i. One of the property-heads of the NN in this project predicts these local atomic energy contributions, which can then summed up to obtain the total energy of the system.

1.8 The *apax* Model Architecture

The *apax* model architecture allows a single network to predict not only different properties, but also allows the prediction of different atomic species. The information about the atomic species is encoded in the learnable parameters.

The forces needed for the MD simulation are calculated via Automatic Differentiation (AD), which allows the forces to be obtained from the energy predictions without the need for a separate force head in the network. The forces are therefore calculated analytically ensuring that they are consistent with the energy predictions, which is crucial for the stability and energy conservation of the MD simulation.

In order to improve the performance of the model, a loss function has to be defined, which quantifies the difference between the predicted properties and the true properties. A common choice for the loss function is the mean squared error (MSE):

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left(w_E (E_i - \hat{E}_i)^2 + w_F \sum_{j=1}^{3N} \| F_{ij} - \hat{F}_{ij} \|^2 \right) \tag{1.10}$$

where N is the number of configurations, E and F are the predicted and true energies and forces, and w_E, w_F are the weights for the energy and forces to the loss function, respectively.

1.9 Uncertainty Quantification

In order to assess the uncertainty of the model at a current simulation step, a shallow ensemble can be used to quantify the uncertainty of the predictions. With a shared backbone in the model but randomly initialized last layers, the ensemble can provide a measure of the uncertainty by evaluating the variance of the predictions across the ensemble members. The uncertainty $sigma_x$ for a property x can be calculated as:

$$\sigma_x = \sqrt{\frac{1}{M-1} \sum_{m=1}^{M} (x_m - \bar{x})^2} \tag{1.11}$$

where M is the number of ensemble members, x_m is the prediction from the m-th ensemble member, and $\bar{x}$ is the mean prediction across the ensemble. This uncertainty can be used to identify configurations where the model is less confident, which can be useful for active learning strategies, where new data points can be added to the training set to improve the model's performance in those regions of the configuration space.

2 Measurements

2.1 Diffusion Coefficient

A quantitative measure of the diffusion of particles in a system is the diffusion coefficient D. Fick's first law relates the flux of particles J to the concentration gradient ∇C through the diffusion coefficient:

$$J = -D\nabla C. \tag{2.1}$$

The negative sign indicates that particles diffuse from regions of higher concentration to regions of lower concentration. The mean squared displacement (MSD) is defined as an ensemble-averaged squared distance that a particle travels over time t. The Einstein relation connects the diffusion coefficient to the MSD:

$$\lim_{t\to\infty} \mathrm{MSD}(t) = 2nDt \tag{2.2}$$

where n is the dimensionality of the system (e.g., $n = 3$ for three-dimensional space). In practice we calculate the MSD by finding the slope m of the mean squared displacement as a function of time and then using the relation:

$$D = \frac{m}{2n} \tag{2.3}$$

The velocity autocorrelation function (VACF) is another tool to analyze the dynamics of particles in a system.

$$D = \frac{1}{3}\int_0^\infty \langle \mathbf{v}(0)\cdot\mathbf{v}(t)\rangle dt \tag{2.4}$$

The diffusion coefficient can be calculated by using the Stokes-Einstein relation, which relates the diffusion coefficient to the temperature T and the viscosity η of the medium:

$$D = \frac{k_B T}{6\pi\eta r} \tag{2.5}$$

where k_B is the Boltzmann constant, and r is the radius of the diffusing particle. This relation assumes that the particle is spherical and that the medium is a continuous fluid with a well-defined viscosity.

2.2 Radial Distribution Function

The Radial Distribution Function (RDF), can be used to analyze the structure of a system by providing information about the probability of finding a particle at a certain distance from a reference particle. The RDF $g(r)$ is defined as:

$$g(r) = \frac{\rho(r)}{\rho_0} \tag{2.6}$$

where $\rho(r)$ is the instantaneous density at a distance r from a reference particle, and ρ_0 is the average density of the system. A RDF value of $g(r) = 1$ indicates that the density at distance r is equal to the average density, while values greater than 1 indicate a higher density (i.e., more particles are found at that distance), and values less than 1 indicate a lower density (i.e., fewer particles are found at that distance). A correct RDF is crucial for accurately for assessing a system, if the RDF is not correct but other properties are, chances are that the correct properties are just a result of error

cancellation and the model is not generalizing well to unseen data.

3 Workflow

3.1 *IPSuite* and *ZnTrack*

The project is completely structured with the tools from *IPSuite* and *ZnTrack*. The former is used to manage the project structure, while the latter is used to track the data and results of the project. The main advantage of using these tools is the node based workflow and the easy data version control. This allows easy reproducibility and scalability of the project. Each step, from data generation by foundation model or classical force-field MD, data relabeling with DFT calculations and dispersion correction, the training of an individual model and the evaluation of the production simulations, is tracked with a node in *ZnTrack* and written in a single workflow.

3.2 From string to box

In order to run a sample simulation, we need to create a box filled with Methanol molecules. The *Smiles2Conformers* node can create the neccessary *ASE* object from a SMILES string. A SMILES (Simplified Molecular-Input Line-Entry System) string is a easy way to represent a molecule in a text format. The *MultiPackmol* node can then be used to pack the molecules into a box according to the desired density. After a geometry optimization, the box is ready to be used for the sample simulation.

3.3 Sample simulation

In order to create new, unseen data to later train the model on, we run a sample simulation. If no prior data is available, a classical force-field taylored for the system can be used to run a short MD simulation. If no fitting force-field is available, a foundation model can be used to generate the forces and energies for the simulation. In this project, we use the *MACE-MP0* model, which is a pre-trained model on the OC20 dataset. The sample simulation is run for 1 nanosecond and 600 frames are saved for later use. The sample frames are chosen randomly.

3.4 Energy convergence via *mgrid* parameter

To ensure a good convergence of the energies, we can vary different parameters of the DFT calculations. Typical parameters to change are the plane-wave cutoff, the grid spacing or even the exchange functional itself. In this project, we choose to vary the grid cutoff value. After running the DFT calculations with different grid cutoff values for the same frame, we can plot the total predicted energy as a function of the grid cutoff value. After that a tradeoff between accuracy and computational cost can be made to choose the optimal grid cutoff value for the rest of the calculations.

3.5 Data relabeling

After the best parameters for the DFT calculations are chosen, the data can be relabeled. To include dispersion interactions, the Grimme D3 correction is added to the DFT energies. The forces are not corrected, as the dispersion interactions are not included in the forces. The relabeled data can then be used to train the model.

3.6 Hyperparameter evaluation

Because the training and precdition of the model is heavily dependent on the choice of hyperparameters, a hyperparameter evaluation is performed. In this project we train several models with different parameters for the network architecture, the cutoff radius and the number of basis functions. The models are then evaluated on a test set and the best model is chosen for the production simulations.

3.7 Final model and production simulation

With the best model chosen, two final models will be trained. One with the original DFT energies and forces, and one with the DFT energies corrected with the Grimme D3 correction. The production simulations are then run for 10 nanoseconds and the results are analyzed to compare the two models and to evaluate the performance of the final model.

3.8 Evaluation

The properties of the self-diffusion coefficient and the Radial Distribution Function (RDF) are calculated from the production simulations and compared to experimental values.

4 Results

4.1 *mgrid* convergence

The *mgrid* parameter was varied from 200 until 500 in steps of 10 in order to find a fitting parameter which has an energy deviation of ≤ 0.005units to its next higher cutoff.

5 Conclusion

6 Summary

7 References